AI Task Orchestrator

1. System Architecture

The **AI Task Orchestrator** is designed to process meeting transcripts and WBS (Work Breakdown Structure) data to extract actionable tasks, assign priorities, and detect scheduling conflicts. The architecture has three main layers:

1. **Frontend (UI Layer)**
    - Built with HTML/CSS.
    - Provides forms for:
        - Pasting a raw meeting transcript.
        - Uploading a WBS CSV file.
    - Displays results in a clean, visual dashboard:
        - Task cards with priority color coding.
        - Conflict highlighting for tasks with logical or scheduling issues.
2. **Backend (Flask API Layer)**
    - Python Flask server handles form submissions.
    - Core responsibilities:
        - Send the meeting transcript to the LLM for task extraction.
        - Parse the LLM's JSON output.
        - Validate WBS CSV for logic conflicts (finish-before-start, missing predecessors).
        - Aggregate stats (High/Medium/Low priority, number of conflicts).
3. **LLM Integration Layer (Gemini AI)**
    - Uses google.generativeai with the **Gemini 2.5 flash model**.
    - Responsibilities:
        - Understand raw text in context.
        - Extract structured tasks with assignees, deadlines, and conflict flags.
        - Score priorities based on urgency/impact.
        - Identify contradictory information in the transcript.

Architecture Diagram (conceptual):

```
[User Interface] --> [Flask Backend] --> [Gemini LLM]
                                |
      |------> [WBS CSV Analysis] <------|
```

---

2. LLM Prompt Engineering

The **LLM prompt** is carefully designed to ensure:

1. **Structured Output:** Forces the model to return only valid JSON.
2. **Detailed Extraction:** Each task must include:
    - task → Description of the task.
    - assignee → Who is responsible.
    - deadline → Due date in YYYY-MM-DD format.
    - priority_score → Number between 1–10.
    - priority → HIGH, MEDIUM, or LOW.
    - conflict_flag → True/False if conflicting information exists.
    - conflict_reason → Explanation if there is a conflict.
3. **Conflict Detection Logic:** The model is prompted to detect:
    - Two people assigning different deadlines to the same task.
    - Overlapping responsibilities that may cause confusion.
    - Contradictions in instructions provided in the meeting transcript.

Example Prompt Logic (Simplified):

```
Extract action items from this transcript.

Return ONLY valid JSON:

[
  {
    "task":"Task description",
    "assignee":"Person",
    "deadline":"YYYY-MM-DD",
    "priority_score": 1-10,
```

"priority":"HIGH or MEDIUM or LOW",

"conflict_flag": true/false,

"conflict_reason":"Reason if conflict"

}

]

Transcript:

[Insert raw meeting transcript here]

How the Gemini LLM works:

1. Reads the transcript **contextually**, identifying tasks, responsible individuals, and deadlines.
2. Evaluates the **urgency and impact** of each task to assign a priority_score.
3. Checks for **conflicts** in the transcript:
    - e.g., Alice says a report is due on March 10, Bob says March 12 → conflict_flag: true.

The output is structured JSON, which can be directly used to populate your dashboard.

---

3. Processing Steps

1. **Input: Raw Meeting Transcript**
    - Paste or upload a transcript in the frontend.
    - Example snippet:
    - Alice will prepare the project proposal by March 10.
    - Bob will complete the design document by March 15.
    - Alice also mentioned the proposal must be reviewed by March 12.
2. **LLM Extraction**
    - Gemini AI reads the transcript.
    - Extracts each **task, assignee, and deadline**.
    - Flags conflicts where dates or instructions contradict.
3. **Priority Assignment**
    - The model calculates a **Priority Score (1–10)** based on urgency/impact.
    - Categorizes as HIGH, MEDIUM, or LOW.

4. **Conflict Detection**
    - If multiple people mention conflicting deadlines or overlapping tasks → conflict_flag: true.
    - Provides a conflict_reason for transparency.
5. **WBS CSV Validation (Optional)**
    - Upload a WBS CSV with task IDs, start/finish dates, and predecessors.
    - Backend checks:
        - Finish_Date < Start_Date → logical error.
        - Predecessor references that do not exist → conflict.
6. **Output**
    - Tasks displayed as cards with:
        - Assignee, deadline, priority, score bar.
        - Highlighted conflicts.
    - WBS conflicts shown in a separate section.

---

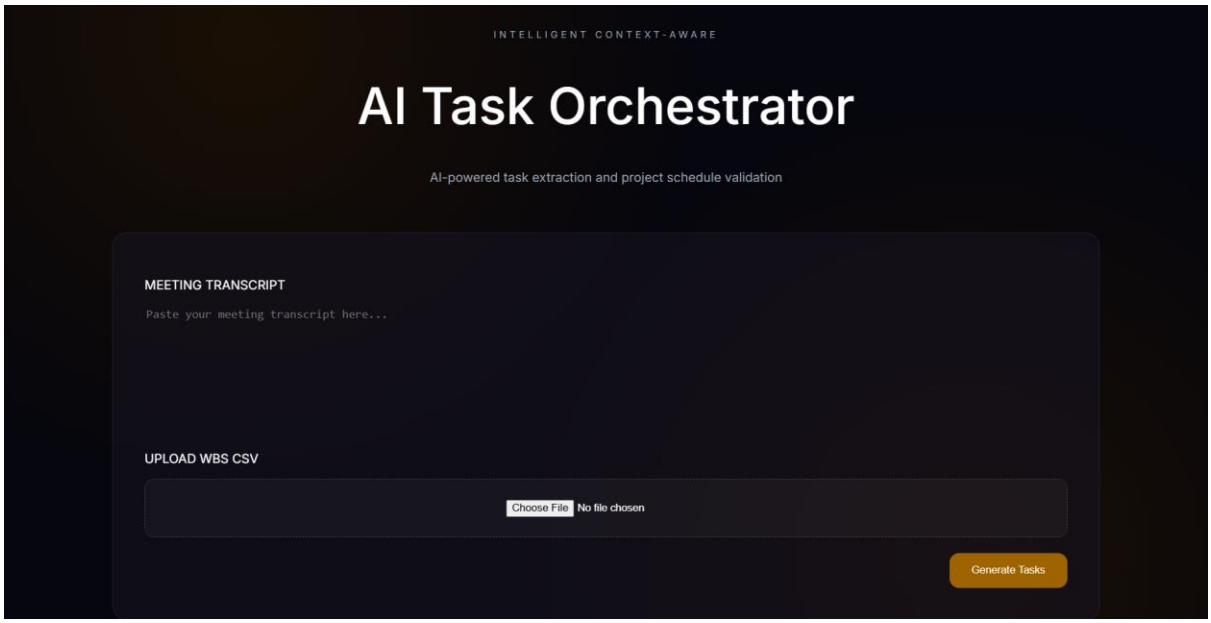
4. Instructions to Run the Project

1. **Clone the repository** (or save the code files).
2. **Install dependencies:**
3. pip install flask pandas google-generativeai
4. **Set your Gemini API Key:**
5. import google.generativeai as genai
6. genai.configure(api_key="YOUR_API_KEY")
7. **Start the Flask app:**
8. python app.py
9. **Open in browser:** http://127.0.0.1:5000/
10. **Use the app:**
    - Paste your **meeting transcript**.
    - Optionally upload a **WBS CSV** to check schedule logic.
    - Click **Generate Tasks** to view structured tasks and conflict analysis.

Key Points:

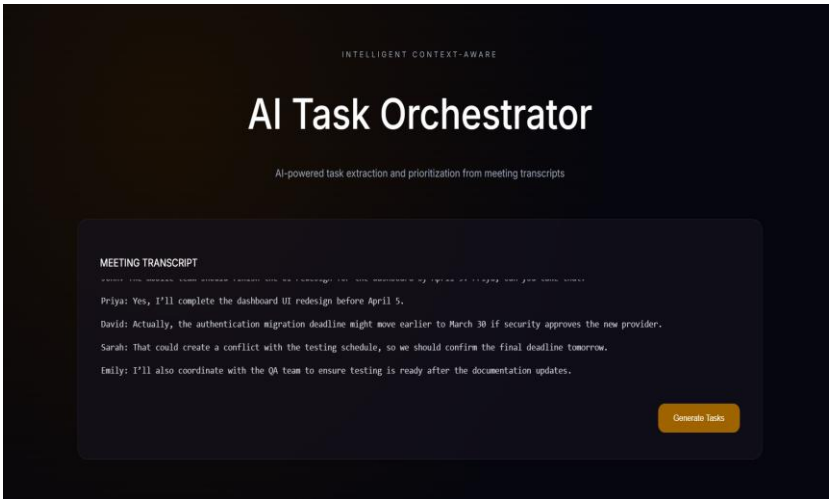
- Gemini LLM provides **context-aware extraction**.
- Prompt engineering ensures **accurate, structured JSON** output.
- WBS analysis detects **schedule inconsistencies** automatically.

Out Put:

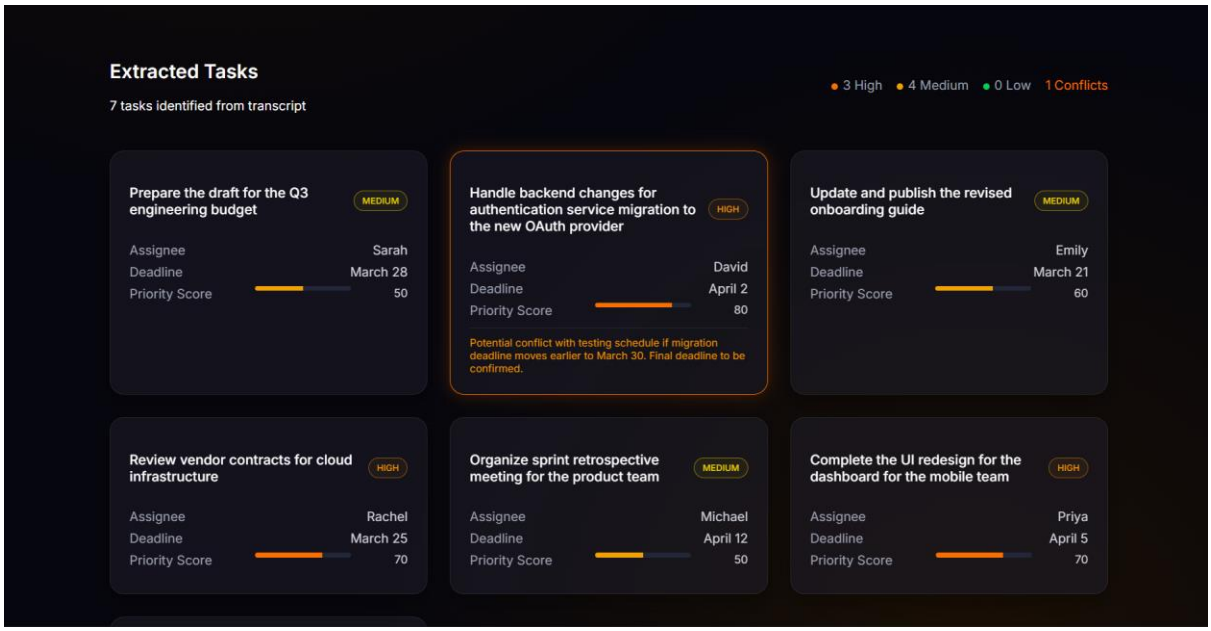


Example:

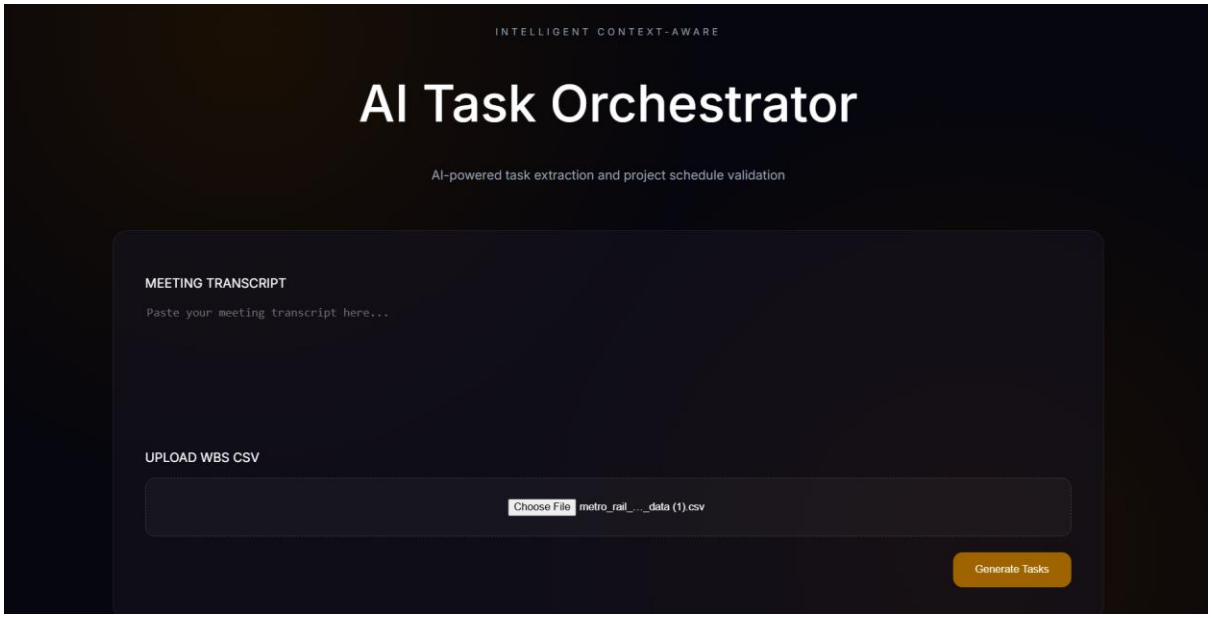
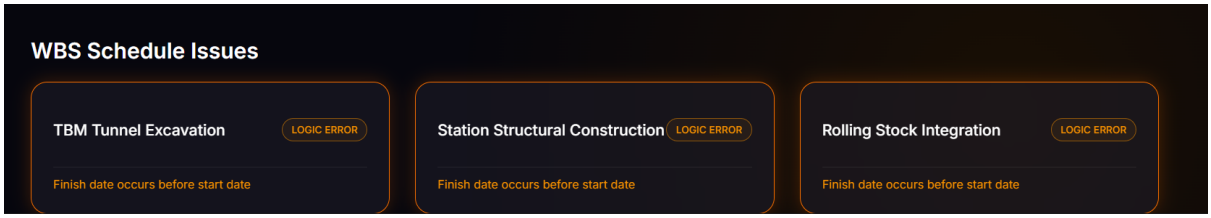
Raw Transcript:



O/P:



WBS FILE:



O/P: